TABasco: Task-Aware Browser Tab Optimization using Reinforcement Learning

Aarussh Shrivastava (B24CM1001) | IIT Jodhpur
CSL3090 Artificial Intelligence, Assignment 2 | October 2026
https://github.com/Aarsussh/TABasco

Abstract

TABasco is an original synthetic browser-tab management MDP trained with a Deep Q-Network (DQN) [1]. In an initial soft-budget evaluation, DQN earned higher composite return while violating the memory budget more often. Increasing the penalty reduced but did not remove violations, so a constraint-aware transition variant was trained. On the previously examined 60-session evaluation block, it maintained the simulator's active-memory invariant. DQN had higher composite return than suspension baselines, although those baselines retained a larger fraction of useful revisits. These results describe the simulator and reward, not a real browser.

1. MDP and simulator

For N=12 slots, state s has 8N+5=101 float32 values. Each slot records occupancy, active status, normalized memory, recency, access frequency, current-task match, estimated revisit probability, and unsaved-work risk. Five global values encode active memory/budget, task, remaining horizon, elapsed fraction, and occupancy. Tab sizes range from 0.45 to 1.35 synthetic units. Tasks switch; tabs arrive, age, expire, and may be revisited. Revisit probability changes with age, frequency, and task affinity.

There are 3N+1=37 actions: KEEP, SUSPEND, or CLOSE for one slot, plus WAIT. KEEP retains an active tab or restores a suspended one; active KEEP is a charged no-op. SUSPEND requires an active tab, CLOSE requires occupancy, and WAIT is always valid. Empty-slot actions and redundant suspension are invalid. Each decision advances one step; episodes end at a fixed 120-step horizon, not when all tabs close.

Only active tabs count against B=8; suspension frees the full tab size and suspended metadata costs zero active memory. Suspended revisits restore the tab and charge restoration. Closed pages retain 30-step history; recovery charges restoration and lost-work costs. Expiration charges risk-weighted loss and retains recovery history. All future revisit and arrival events are sampled from the evolving session state.

2. Reward and constraint handling

Reward is the negative sum of positive costs: $r = -[c_m \max(0,M-B) + C_{restore} + 1.2\, C_{lost} + 0.45\, C_{disrupt} + C_{action} + C_{deferred}]$. The selected $c_m=1$; restoration multiplier is 0.8, ordinary action/WAIT cost is 0.035, invalid action cost is 0.12, and constrained-event deferral cost is 0.25. Per-step reward components sum to the returned reward. There is no tab-count bonus or terminal reward.

The penalty sweep (coefficients 1, 3, 5; two fresh training seeds each) left 11.18%, 9.89%, and 8.59% of validation steps over budget. Stronger penalties alone were insufficient. In the selected constraint-aware variant, initial overflow tabs start suspended; infeasible restores are masked; suspended revisits that cannot fit stay suspended; closed recovery remains in history if it cannot fit; and over-budget arrivals are declined, not queued. Every transition checks active memory after all automatic events. The guarantee is specific to this simulator's active-memory model.

3. DQN and baselines

The DQN is a 101-128-128-37 ReLU MLP with Adam (0.001), discount 0.97, batch size 64, replay capacity 50,000, Huber loss, gradient clipping at 10, and target updates every 500 learner updates. Epsilon decays from 1.0 to 0.05 over 20,000 decisions. The same observation-derived action mask is applied in exploration, greedy selection, and replay bootstrap. Runs used CPU, 250 episodes of up to 120 steps, and fresh network seeds 3090 and 4090.

LRU suspends the eligible active tab with the oldest last visit; LFU uses the lowest access count then oldest; Random samples uniformly from this same eligible set. All WAIT when no active tab is eligible. They are evaluation baselines, not training labels. Policies share the same initial scenario and event seed per episode; DQN is greedy at evaluation.

4. Design evolution and model selection

The first version explored empty-slot actions without masking, could end when all tabs closed, and did not account consistently for no-op and expiry costs. Corrected runs use one validity mask, a fixed horizon, and explicit reward-component checks. The soft-budget evaluation still exceeded capacity because arrivals and automatic returns could increase memory after the agent decision; raising the penalty from 1 to 5 only reduced validation violation frequency from 11.18% to 8.59%.

Constraint-aware transitions were introduced to meet the explicit capacity requirement while making infeasible returns visible as deferrals. A later reward-weight experiment did not support replacing the selected model: disruption-weight 0.35 changed paired return by +2.43 for training seed 3090 and -1.47 for seed 4090; revisit success declined. The two-seed average is descriptive and uncertain.

Policy update

The finite-horizon objective maximizes expected discounted return, E[sum gamma^t r_t], with gamma=0.97. Replay stores state, action, reward, next state, terminal flag, and next-state action mask. DQN minimizes Huber loss against target r + gamma(1-done) max valid-action Q-target(next state). A target network supplies bootstrap values; epsilon-greedy exploration samples valid actions only [1,2].

Transition sequence

After the management action, time advances, the task may switch, and revisit probabilities update. Bernoulli draws determine candidates; at most one open-tab revisit is selected, preferring the highest current probability. Closed-history recovery may then occur, followed by at most one new arrival and stochastic age-based expiration. Reward uses final active memory. In soft mode, automatic restoration or arrival can exceed the budget after the decision; a tab-by-tab policy cannot preempt that event. Constraint-aware mode instead defers or declines infeasible events and checks the post-event memory bound.

5. Measured results

The selected checkpoint evaluation used seeds 91000-91059. This block was examined during earlier project analysis, so it is a previously examined evaluation, not a newly untouched test. Table 1 reports scenario mean and SD.

Policy	Return +/- SD	Mem.	Useful	Viol.
DQN 3090	-21.70 +/- 4.51	5.50	0.824	0.00
DQN 4090	-16.31 +/- 3.52	5.80	0.849	0.00
LRU	-58.38 +/- 4.67	1.17	0.886	0.00
LFU	-58.43 +/- 4.67	1.18	0.886	0.00
Random	-58.55 +/- 4.62	1.25	0.887	0.00

Table 1. Common-seed policy comparison; useful is task-matching revisit success.

Every policy had zero violation steps and zero invalid actions under constraint-aware rules. DQN useful-revisit success was 0.824/0.849, below LRU/LFU/Random at 0.886/0.886/0.887. DQN restoration costs were 5.69/4.37 and disruption costs 3.67/2.07; baselines paid restoration 16.32-16.35 and disruption 36.17-36.30. DQN lost-work costs were higher (6.73/4.33 versus about 1.69), and deferred-event costs were 1.41/1.34. Return therefore reflects a reward-weighted tradeoff, not universal superiority.

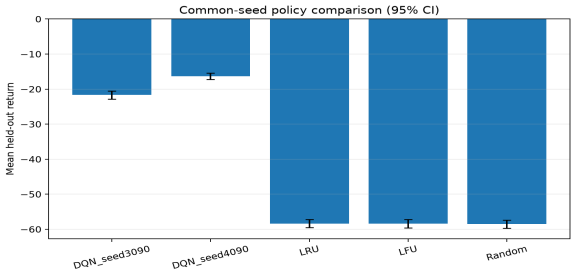


Figure 1. Mean return and 95% scenario confidence intervals.

Reward cost profile

Policy	Restore	Lost	Disrupt	Defer
DQN 3090	5.69	6.73	3.67	1.41
DQN 4090	4.37	4.33	2.07	1.34
LRU	16.32	1.69	36.17	0.00
LFU	16.35	1.69	36.19	0.00
Random	16.34	1.70	36.30	0.01

Mean episode cost components; memory cost is zero for every policy under the hard constraint; action cost is 4.20 each.

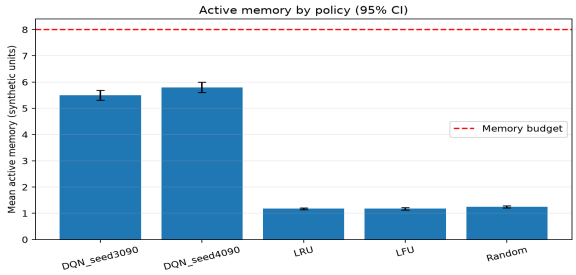


Figure 2. Mean active memory and 95% intervals; budget is 8.

6. Interpretation and limits

The later reward-weight validation did not justify replacing the selected model. Lowering the disruption coefficient to 0.35 changed paired canonical return by +2.43 for training seed 3090 (95% scenario CI +0.26 to +4.59) and -1.47 for seed 4090 (-3.43 to +0.49), mean +0.48 over only two training seeds. Useful revisit fell from 0.820/0.834 to 0.797/0.802. The effect reversed by seed and remained uncertain; current checkpoints were retained. The prior evaluation block did not inform that validation choice.

Capacity safety is enforced by the environment, not learned by DQN. The workload and memory units are synthetic; scenario intervals do not measure training-seed uncertainty. More seeds, calibrated workloads, and memory measurements are needed before deployment claims.

References

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529-533, 2015. https://doi.org/10.1038/nature14236.

[2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.